Hacettepe University - Computer Engineering

BBM204 Software Practium II - Spring 2025

PROGRAMMING ASSIGNMENT II

Programming Assignment 2

MediPlan: Intelligent Diagnostic Planner

Topics: Dynamic Programming

Programming Language: Java (OpenJDK 11)
Due Date: **Monday, 06/04/2026 (23:59:59)** over
https://submit.cs.hacettepe.edu.tr

1 Introduction

The year is 1978. Hacettepe University Hospital has not been open for long, and like most hospitals of its era, it has no unified laboratory information system. When a physician orders a diagnostic workup, each test is requested independently; a blood draw for cholesterol here, another for creatinine there, a urine sample ordered separately with no knowledge of what has already been collected from the same patient that morning. In a busy ward, the same sample is drawn twice, the same analysis are run twice, the same patient is burdened twice, because no one has a complete picture of what has already been done.

Students from the newly opened Computer Engineering Department of the Hacettepe University has been given a task: build the first automated diagnostic planning system. Given a catalogue of diagnostic tests and the relationships between them, the system must determine the most efficient way to evaluate any requested clinical score, allowing for minimizing both the number of distinct collection procedures the patient must undergo and the total processing cost to the hospital.

Some tests measure something directly from the patient, for example a blood draw, a urine sample, a blood pressure reading. Others are not collected at all but *computed*: a cholesterol ratio derived from two blood measurements, a kidney filtration rate derived from a creatinine level, a cardiovascular risk score aggregated from several derived indicators.

This hierarchy of raw tests feeding into derived indicators feeding into composite scores forms a **dependency graph**. And like all dependency graphs, it has a critical inefficiency hiding inside it: if two different composite scores both depend on the same underlying test, a naive system would perform that test multiple times. In 1978, with no electronic coordination between physicians, that is not a hypothetical. It happens every day.

MediPlan is the system that will change this. Given a diagnostic catalogue defining how tests depend on each other, MediPlan computes two things:

- **The minimum patient burden**: How many distinct physical procedures (blood draw, urine sample, etc.) the patient must undergo to support a requested diagnostic score. Minimizing this directly minimizes patient discomfort.

- **The hospital cost under two models**:
  - A **naive aggregated cost**, where each dependency chain is evaluated independently and shared subtrees may be counted multiple times.
  - An **optimized execution cost**, where each test is performed at most once and its result is reused wherever needed.

  Minimizing the optimized cost directly minimizes resource expenditure.

These objectives are evaluated on the same dependency graph but require different algorithmic strategies, including two different dynamic programming strategies.

Note: Dynamic Programming

Dynamic programming solves problems by breaking them into overlapping subproblems and storing the results so they are never recomputed. A system that recomputes creatinine levels every time a derived indicator needs them has forgotten the past.

"Those who cannot remember the past are condemned to repeat it."
— Dynamic Programming (slide 22)

2 Problem Definition

2.1 Laboratory Test Types

The diagnostic catalogue contains three types of tests:

Type	Definition	Cost
`RAW`	Collected directly from the patient via a physical procedure. Has no dependencies on other tests.	Explicit `collection_cost` attribute in XML (integer). Represents lab processing time and equipment cost.
`DERIVED`	Computed from one or more `RAW` tests. Has no explicit cost in the XML.	Computed by your system in Step 2.
`COMPOSITE`	Aggregated from one or more `DERIVED` or `COMPOSITE` tests to produce a clinical score. Never depends directly on `RAW` tests. Has no explicit cost in the XML.	Aggregations of costs of their dependencies.

Each `RAW` test also carries a `sample_type` field indicating what physical procedure is required to collect it. The sample types in this assignment are: `BLOOD`, `URINE`, `TISSUE`, and `NONE` (for non-invasive measurements such as blood pressure).

2.2 The Two Optimization Objectives

Objective 1: Patient Burden (solved by top-down DP in Step 3)

The patient burden of a diagnostic test is the number of distinct non-`NONE` sample types in its full dependency subtree. It answers the question: *How many different physical procedures must this patient undergo?* Non-invasive measurements (sample type `NONE`) do not count toward the burden since they require no collection procedure.

Method:

$$\text{burden}(t) = \text{number of distinct non-NONE collection procedures required to compute } t$$

Example: if computing `cardiovascular_risk` requires tests that need `BLOOD` and `URINE` samples, the patient burden is 2 (regardless of how many individual tests require those samples).

Objective 2: Hospital Cost

Two cost values are computed across Steps 4 and 5:

Naive aggregated cost (Step 4, bottom-up DP): The cost obtained by evaluating the dependency structure as defined, where each test contributes to every higher-level test that depends on it. Shared subtrees are therefore computed once but counted multiple times if necessary.

$$\text{naiveCost}(t) \;=\; t.\text{cost} \;+\; \sum_{d \,\in\, \text{inputs}(t)} \text{naiveCost}(d)$$

Optimized execution cost (Step 5, traceback): The true minimum cost, where each test is counted at most once:

$$\text{optimizedCost}(\text{target}) \;=\; \sum_{t \,\in\, \text{executionPlan}(\text{target})} t.\text{cost}$$

Step 4 uses bottom-up DP to compute the naive aggregated cost efficiently for multiple targets at once. Step 5 eliminates the redundancy at the execution level by constructing a plan where each test is referenced only once. Dynamic programming in Step 4 ensures that shared dependencies are not recomputed multiple times, but their costs are still aggregated separately for each dependency chain. Step 5 eliminates this redundancy at the execution level by constructing a plan where each test is performed only once.

2.3 Computing Processing Cost for Derived Tests (Preprocessing)

`RAW` tests have explicit `collection_cost` values in the XML. `DERIVED` tests must have their `processing_cost` computed in Step 2. `COMPOSITE` tests do not have `processing_cost` since they only aggregate their dependencies.

3 Implementation Steps

Step 1: Parse XML and Build Dependency Graph

Read `diagnostic_catalogue.xml` using standard Java XML parsing (`javax.xml.parsers`). No external libraries are needed. Populate a map of test objects keyed by test ID. Build a directed dependency graph where an edge from test A to test B means A depends on B.

No printed output is required for this step. It is tested independently via unit tests on the data structures you populate.

Step 2: Compute DERIVED Test Costs

In `DiagnosticCatalogue.java`, implement `computeDerivedCosts()` function. For each `DERIVED` test, iterate over its direct inputs. Collect their `sample_type` values, count the distinct non-`NONE` ones, and assign that count to the test's `processing_cost` field.

Step 2 examples:

- `cholesterol_ratio` inputs: `total_cholesterol(BLOOD)`, `hdl_cholesterol(BLOOD)`.
  Distinct non-`NONE` types: {`BLOOD`} ⇒ `processing_cost = 1`.
- `kidney_stress` inputs: `creatinine(BLOOD)`, `urine_protein(URINE)`.
  Distinct non-`NONE` types: {`BLOOD`, `URINE`} ⇒ `processing_cost = 2`.
- `liver_function` inputs: `liver_enzyme(BLOOD)`, `biopsy_score(TISSUE)`.
  Distinct non-`NONE` types: {`BLOOD`, `TISSUE`} ⇒ `processing_cost = 2`.
- `bmi_indicator` inputs: `bmi(NONE)`.
  Distinct non-`NONE` types: {} ⇒ `processing_cost = 0`.

This step must complete before Step 3 runs.

Expected output:

```
##COST COMPUTATION##
cholesterol_ratio    processing_cost: 1
kidney_stress        processing_cost: 2
liver_function       processing_cost: 2
bmi_indicator        processing_cost: 0
##COST COMPUTATION COMPLETED##
```

Step 3: Top-Down Memoized DP a.k.a. Patient Burden

In `MediPlanDP.java` implement `computeBurdenTopDown(String targetId)` function.

Given the `single_target` from the requests file, compute the minimum patient burden using top-down dynamic programming with memoization.

The memo table maps each test ID to the **set of sample types** reachable from it.

Recurrence:

$$\text{sampleSet}(t) = \begin{cases} \text{(cached value)} & \text{if } t \text{ is already memoized} \\ \{\} & \text{if } t \text{ is } \texttt{RAW} \text{ and sample_type} = \texttt{NONE} \\ \{\, t.\text{sample_type} \,\} & \text{if } t \text{ is } \texttt{RAW} \text{ and sample_type} \neq \texttt{NONE} \\ \bigcup_{d \,\in\, \text{inputs}(t)} \text{sampleSet}(d) & \text{otherwise} \end{cases}$$

$$\text{burden}(t) = |\,\text{sampleSet}(t)\,|$$

Print each recursive call as it is made. When a memoized result is returned, print it clearly.

Expected output for `single_target = "cardiovascular_risk"`:

```
##PATIENT BURDEN ANALYSIS (Top-Down DP)##
Called computeSamples(cardiovascular_risk)
  Called computeSamples(cholesterol_ratio)
    Called computeSamples(total_cholesterol)
    total_cholesterol -> [BLOOD]
    Called computeSamples(hdl_cholesterol)
    hdl_cholesterol -> [BLOOD]
  cholesterol_ratio memoized -> [BLOOD]
  Called computeSamples(bmi_indicator)
    Called computeSamples(bmi)
    bmi -> {} (NONE, not counted)
  bmi_indicator memoized -> []
cardiovascular_risk memoized -> [BLOOD]

Patient burden for cardiovascular_risk: 1 sample type(s): [
   BLOOD]
##PATIENT BURDEN ANALYSIS COMPLETED##
```

Step 4: Bottom-Up Tabulation DP (Naive Aggregated Hospital Cost)

In `MediPlanDP.java` implement `computeCostBottomUp(List<String> targetIds)`.

Given the `all_targets` list, compute the **naive** hospital cost for each target using bottom-up dynamic programming with tabulation. The naive cost follows each dependency chain independently and shared subtrees may be counted multiple times. This is the baseline that Step 5 will improve upon.

Unlike Step 3 which lazily evaluates a single target, bottom-up eagerly fills a cost table for all reachable tests at once and then answers all target queries by lookup. This is the right choice when *multiple targets* are requested since the table is filled once and any target can be answered in $O(1)$.

Deriving topological order: Run DFS on the dependency graph recording finish times. Reverse finish order gives a valid topological ordering in which all dependencies of a test appear before the test itself.

Recurrence:

$$\text{totalCost}(t) = \begin{cases} t.\text{collection_cost} & \text{if } t \text{ is } \texttt{RAW} \\ t.\text{processing_cost} + \sum_{r \in \text{inputs}(t)} \text{totalCost}(r) & \text{if } t \text{ is } \texttt{DERIVED} \\ \sum_{d \in \text{inputs}(t)} \text{totalCost}(d) & \text{if } t \text{ is } \texttt{COMPOSITE} \end{cases}$$

Expected output for `all_targets = ["cardiovascular_risk"]` using the simple catalogue:

```
##HOSPITAL COST ANALYSIS (Bottom-Up DP)##

Topological order: total_cholesterol, hdl_cholesterol, bmi,
                   cholesterol_ratio, bmi_indicator,
                   cardiovascular_risk

Computing total_cholesterol:         collection_cost = 3
   total: 3
Computing hdl_cholesterol:           collection_cost = 3
   total: 3
Computing bmi:                       collection_cost = 1
   total: 1
Computing cholesterol_ratio:         processing_cost = 1 + 3 +
    3       total: 7
Computing bmi_indicator:             processing_cost = 1
                 total: 1
Computing cardiovascular_risk:       cost = 7 + 1
   total: 8

Results:
  cardiovascular_risk          naive hospital cost: 8
##HOSPITAL COST ANALYSIS COMPLETED##
```

Note on topological order

The topological order shown above is one valid ordering, however any ordering in which all dependencies of a test appear before the test that depends on them is equally correct. Your output may list tests in a different order depending on your DFS traversal. The autograder accepts any valid topological ordering.

Step 5: Traceback: Combined Diagnostic Plan

In `MediPlanDP.java` implement `buildDiagnosticPlan(String targetId)`.

Using results from both Step 3 and Step 4, produce a structured diagnostic plan for the `single_target`.

The plan shows:

- Patient burden and required procedures (from `burdenMemo`)
- Execution order filtered from `topoOrder` to the reachable subtree
- Per-test **added cost** annotation
- Sample type markers for RAW collection tests
- **Naive hospital cost** from `costTable` (the Step 4 baseline)
- **Optimized hospital cost**: sum of added costs across the execution plan, counting each test exactly once, this is the true minimum

Expected output for `single_target = "cardiovascular_risk"` using the simple catalogue:

```
##DIAGNOSTIC PLAN##
Target: cardiovascular_risk (Cardiovascular Risk Score)

Patient burden:  1 sample type(s) required
  Procedures:    BLOOD draw

Execution order (hospital cost optimized):
  [ 1]  total_cholesterol            [RAW      ] added cost:
     3  <- BLOOD sample
  [ 2]  hdl_cholesterol              [RAW      ] added cost:
     3  <- BLOOD sample
  [ 3]  bmi                          [RAW      ] added cost:
     1
  [ 4]  cholesterol_ratio            [DERIVED  ] added cost:
     1
  [ 5]  bmi_indicator                [DERIVED  ] added cost:
     0
  [ 6]  cardiovascular_risk          [COMPOSITE] added cost:
     0

Naive hospital cost:     8  (shared dependencies counted
   multiple times)
Optimized hospital cost: 8  (each test counted once)
##DIAGNOSTIC PLAN COMPLETED##
```

Note on execution order

The execution order is derived from the topological order computed in Step 4. Since any valid topological order is accepted, your execution order may differ from the example above while still being correct. `COMPOSITE` tests always show `added cost: 0` because they carry no processing cost of their own.

The **optimized hospital cost** might differ from the naive cost because it is computed over the reachable subgraph of the target, counting each test exactly *once*, rather than aggregating costs along each dependency chain independently.

4 Input Format

4.1 diagnostic_catalogue.xml

The first command-line argument. An XML file defining the diagnostic catalogue.

Fields for `RAW` tests use XML attributes; non-`RAW` tests list their dependencies as child `<input>` elements.

Fields for `RAW` tests:

```
<?xml version="1.0" encoding="UTF-8"?>
<catalogue>

  <test id="total_cholesterol" name="Total Cholesterol"
        type="RAW" sample_type="BLOOD" collection_cost="3"/>

</catalogue>
```

Fields for `DERIVED` and `COMPOSITE` tests:

```
<catalogue>

  <test id="cholesterol_ratio" name="Cholesterol Ratio" type="
     DERIVED">
    <input ref="total_cholesterol"/>
    <input ref="hdl_cholesterol"/>
  </test>

</catalogue>
```

Note: `DERIVED` tests have no cost attribute; you compute their `processing_cost` in Step 2. `COMPOSITE` tests do not have a `processing_cost` and require no computation beyond aggregation.

4.2 diagnostic_requests.xml

The second command-line argument. Specifies two query types:

```
<?xml version="1.0" encoding="UTF-8"?>
<requests>
  <single_target ref="overall_health"/>
  <all_targets>
    <target ref="overall_health"/>
    <target ref="cardiovascular_risk"/>
    <target ref="diabetes_risk"/>
  </all_targets>
</requests>
```

`single_target` is used for the top-down patient burden analysis (Step 3). `all_targets` is used for the bottom-up hospital cost analysis (Step 4).

4.3 Sample Input

The following is a simplified catalogue used for the examples in this document. The starter code package includes `diagnostic_catalogue_full.xml`, `diagnostic_requests_full.xml`, and `expected_output_full.txt`. These cover all test types (`BLOOD`, `URINE`, `TISSUE`, `NONE`), shared dependencies, and multiple composite targets. Use them to verify your output before submitting.

Listing 1: Simple diagnostic_requests.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<requests>
  <single_target ref="cardiovascular_risk"/>
  <all_targets>
    <target ref="cardiovascular_risk"/>
  </all_targets>
</requests>
```

Listing 2: Simple diagnostic_catalogue.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<catalogue>

  <test id="total_cholesterol" name="Total Cholesterol"
        type="RAW" sample_type="BLOOD" collection_cost="3"/>

  <test id="hdl_cholesterol" name="HDL Cholesterol"
        type="RAW" sample_type="BLOOD" collection_cost="3"/>

  <test id="bmi" name="Body Mass Index"
        type="RAW" sample_type="NONE" collection_cost="1"/>

  <test id="cholesterol_ratio" name="Cholesterol Ratio" type="
     DERIVED">
    <input ref="total_cholesterol"/>
    <input ref="hdl_cholesterol"/>
  </test>

  <test id="bmi_indicator" name="BMI Indicator" type="DERIVED
     ">
    <input ref="bmi"/>
  </test>

  <test id="cardiovascular_risk" name="Cardiovascular Risk
     Score"
        type="COMPOSITE">
    <input ref="cholesterol_ratio"/>
    <input ref="bmi_indicator"/>
  </test>

</catalogue>
```

5 Run Configuration

```
javac *.java -d .
java Main <diagnostic_catalogue.xml> <diagnostic_requests.xml>
```

6 Starter Code Structure

You **must** use the provided starter code. Do not change any class or method name or signature. You may add private helper methods freely.

File	Your task
`Main.java`	Entry point. Provided mostly complete. Fill in the method calls in order.
`DiagnosticCatalogue.java`	Implement: `loadFromXML()`, `computeDerivedCosts()`. Starter provides the `Test` inner class and field declarations.
`MediPlanDP.java`	Implement: `collectReachable()`, `computeBurdenTopDown()`, `computeCostBottomUp()`, `topologicalSort()`, `dfsFinish()`, `buildDiagnosticPlan()`. Starter provides memo table, cost table declarations, and `getExecutionOrder()` in full.
`DiagnosticRequest.java`	Provided in full. Do not modify. Parses `diagnostic_requests.xml`.

Starter code

The starter code is available at: Starter Code
Submit via `https://submit.cs.hacettepe.edu.tr/`

7 Grading Policy

Component	Step	Weight
XML parsing + graph construction (Steps 1 & 2)	1, 2	10%
Top-down memoized DP (patient burden)	3	30%
Bottom-up tabulation DP (naive cost)	4	35%
Traceback + optimized cost	5	15%
Overall output format test	—	10%
Total		**100%**

8 Important Notes

- **Brute force solutions that do not use memoization or tabulation will receive 0 for Steps 3 and 4.**
- Both DP implementations must produce consistent results. The `burdenMemo` populated in Step 3 and the `costTable` populated in Step 4 must both be complete and correct before Step 5 runs.
- Input is guaranteed to be a valid so no circular dependencies will appear in test inputs.
- Do not use any external libraries. The input files use XML, which is parsed using the standard Java `javax.xml.parsers` library.
- Do not use packages. Place all `.java` files directly in your submission zip.
- Do not submit `.jar` or `.class` files.
- **Deadline: Monday, 06.04.2026 at 23:59:59.**
- Test your code using the TurBo Grader before submitting (does not count as submission).
- Save all your work until the assignment is graded.
- You are responsible for all announcements made on Piazza.

Submit via https://submit.cs.hacettepe.edu.tr/ with the following file hierarchy:

```
b<studentID>.zip
  Main.java
  DiagnosticCatalogue.java
  MediPlanDP.java
  DiagnosticRequest.java
  *.java                          (optional helper classes)
```

9 Academic Integrity Policy

All work on assignments **must be done individually**. You are encouraged to discuss the given assignments with your classmates, but these discussions should be carried out in an abstract way. That is, discussions related to a particular solution to a specific problem (either in actual code or in pseudocode) **will not be tolerated**. In short, turning in someone else's work (including work available on the internet), in whole or in part, as your own will be considered as **a violation of academic integrity**. Please note that the former condition also holds for the material found on the web as everything on the web has been written by someone else.

The submissions will be subjected to a similarity check. Any submissions that fail the similarity check will not be graded and will be reported to the ethics committee as a case of academic integrity violation, which may result in the suspension of the involved students. Do not copy any code produced by LLM tools (e.g., ChatGPT, Gemini, Copilot) or any online source.

Disclaimer

The TAs responsible for this assignment are computer engineers, not physicians. All diagnostic tests, costs, and clinical scores are fictional constructs designed to illustrate dependency graphs and dynamic programming. Hacettepe University assumes no liability for any medical decisions made on the basis of this assignment.